

Relatório de Elaboração do Projeto

Sistema de Gestão de Reservas de Hotel

Formando: Nelson Proença

Formador: Carlos Barata

UFCD: 10790 – Projeto de Programação

Data: 02/06/2026

Índice

1. Introdução	4
Contextualização	4
Objetivo do Relatório	4
2. Descrição do Projeto	5
Visão Geral	5
Escopo.....	5
Incluído no projeto:.....	5
Não incluído:	5
Objetivos.....	5
3. Levantamento de Requisitos	6
Metodologia de Recolha	6
Requisitos Funcionais.....	6
Requisitos Não Funcionais	7
4. Análise de Sistemas.....	8
Casos de Uso	8
Análise de Riscos	8
5. Design do Sistema	9
Arquitectura do Sistema	9
Design de Componentes	9
Base de Dados	9
6. Implementação.....	10
Cronograma — Diagrama de Gantt (25 horas).....	10
Tecnologias Utilizadas.....	10
Estrutura de Código.....	10
Controlo de Versão.....	11
7. Testes	12
Plano de Testes.....	12
Casos de Teste e Resultados.....	12
8. Implantação.....	13
Plano de Implantação	13
Formação	13
Suporte	13
9. Conclusão	14
Resumo dos Resultados.....	14
Próximos Passos	14
Reflexão	14

10. Anexos	15
------------------	----

1. Introdução

Contextualização

Este relatório documenta o desenvolvimento do projeto “Sistema de Gestão de Reservas de Hotel”, desenvolvido no âmbito da UFCD 10790 – Projeto de Programação. A necessidade deste sistema surgiu da problemática da gestão manual das reservas num hotel, que pode causar erros como marcações duplicadas, dificuldades na consulta da disponibilidade dos quartos e atrasos no atendimento aos clientes.

Objetivo do Relatório

O objetivo deste relatório é apresentar todas as etapas de elaboração do projeto, desde o levantamento de requisitos até à implementação e testes, fornecendo uma visão detalhada do progresso e das decisões tomadas ao longo do desenvolvimento.

2. Descrição do Projeto

Visão Geral

O Sistema de Gestão de Reservas de Hotel em Python visa automatizar e otimizar as operações diárias de um hotel, nomeadamente o registo de clientes, a gestão de quartos, a efectuação e cancelamento de reservas, os processos de check-in e check-out, e a geração de relatórios de ocupação.

Escopo

Incluído no projeto:

- Registo e gestão de clientes
- Gestão de quartos (CRUD completo)
- Efectuação e cancelamento de reservas
- Registo de check-in e check-out
- Consulta de disponibilidade de quartos
- Histórico de reservas por cliente
- Relatório de ocupação do hotel
- Autenticação de utilizadores com perfis (rececionista, gestor, administrador)

Não incluído:

- Interface gráfica (GUI)
- Gestão de pagamentos e facturas
- Integração com sistemas externos
- Aplicação web ou mobile

Objetivos

- Automatizar o processo de reservas e eliminar erros de marcações duplicadas
- Permitir a consulta rápida da disponibilidade dos quartos
- Registar e controlar os processos de check-in e check-out
- Fornecer relatórios de ocupação para apoio à gestão do hotel
- Garantir a segurança dos dados dos clientes e utilizadores

3. Levantamento de Requisitos

Metodologia de Recolha

Os requisitos foram recolhidos com base na análise do problema identificado na Tarefa 1 do projeto, considerando as necessidades de gestão de um hotel: organização das reservas, controlo da disponibilidade dos quartos e gestão do histórico de clientes. O documento de levantamento de requisitos foi elaborado em formato Excel (UFCD 10790 – Levantamento de Requisitos, versão 1.0, de 02/06/2026).

Requisitos Funcionais

ID	Nome	Descrição	Ator	Prioridade
RF01	Registo de Clientes	O sistema deve permitir registar um novo cliente com nome, contacto, email e documento de identidade.	Rececionista	Alta
RF02	Consulta de Disponibilidade	O sistema deve permitir consultar os quartos disponíveis para um intervalo de datas especificado.	Rececionista / Cliente	Alta
RF03	Efetuar Reservas	O sistema deve permitir criar uma reserva associando um cliente, um quarto e as datas de check-in e check-out.	Rececionista	Alta
RF04	Cancelar Reservas	O sistema deve permitir cancelar uma reserva existente, libertando o quarto para o período correspondente.	Rececionista	Alta
RF05	Registo dos Check-in	O sistema deve registar a entrada do cliente no hotel, actualizando o estado da reserva.	Rececionista	Alta
RF06	Registo dos Check-out	O sistema deve registar a saída do cliente, calcular o valor total da estadia e actualizar o estado do quarto.	Rececionista	Alta
RF07	Lista de Reservas	O sistema deve listar todas as reservas com filtros por data, estado e cliente.	Rececionista / Gestor	Alta
RF08	Gestão de Quartos	O administrador deve poder adicionar, editar e remover quartos, definindo tipo, número, preço por noite e estado.	Administrador	Alta
RF09	Histórico de Reservas	O sistema deve permitir consultar o histórico completo de reservas de um cliente específico.	Rececionista / Gestor	Média
RF10	Relatório de Ocupação	O sistema deve gerar um relatório de ocupação do hotel para um período definido, indicando a taxa de ocupação.	Gestor	Média

RF11	Autenticação de Utilizadores	O sistema deve autenticar os utilizadores com nome de utilizador e password, atribuindo o perfil correcto.	Todos	Alta
------	------------------------------	--	-------	------

Requisitos Não Funcionais

ID	Nome	Descrição	Ator	Prioridade
RNF01	Desempenho	O sistema deve responder a qualquer operação em menos de 2 segundos em condições normais.	Todos	Alta
RNF02	Segurança de Passwords	As passwords devem ser armazenadas com hash (SHA-256), nunca em texto simples.	Sistema	Alta
RNF03	Prevenção de Duplicação	O sistema deve impedir a criação de reservas duplicadas para o mesmo quarto no mesmo período.	Sistema	Alta
RNF04	Usabilidade	A interface deve ser clara e utilizável por um rececionista sem formação técnica em informática.	Rececionista	Média
RNF05	Fiabilidade dos Dados	O sistema deve manter a consistência dos dados usando transações na base de dados.	Sistema	Alta
RNF06	Aperfeiçoamento	O código deve estar organizado em camadas (DAL/BLL) e documentado para facilitar futuras alterações.	Programador	Média
RNF07	Compatibilidade	A aplicação deve funcionar em Windows 10/11 com Python 3.10 ou superior.	Todos	Média

4. Análise de Sistemas

Casos de Uso

- **Registrar Cliente:** A rececionista regista os dados de um novo hóspede no sistema.
- **Consultar Disponibilidade:** A rececionista verifica quais os quartos disponíveis para um período.
- **Efetuar Reserva:** A rececionista associa um cliente a um quarto para um período específico.
- **Cancelar Reserva:** A rececionista cancela uma reserva existente, libertando o quarto.
- **Fazer Check-in:** A rececionista regista a entrada do cliente no hotel.
- **Fazer Check-out:** A rececionista regista a saída do cliente e o valor a pagar.
- **Gerir Quartos:** O administrador adiciona, edita ou remove quartos do sistema.
- **Consultar Relatório:** O gestor consulta a taxa de ocupação e receita do hotel.

Análise de Riscos

Risco	Probabilidade	Mitigação
Reservas duplicadas para o mesmo quarto	Alta	Validação na BD antes de gravar — consulta SQL que verifica sobreposição de datas.
Perda de dados por erro durante uma operação	Média	Uso de transações SQLite com commit para garantir consistência.
Acesso não autorizado ao sistema	Média	Autenticação com hash de password e perfis de utilizador diferenciados.
Erros de formato nas datas introduzidas	Alta	Validação do formato YYYY-MM-DD com tratamento de exceções.

5. Design do Sistema

Arquitectura do Sistema

O sistema foi desenvolvido seguindo a arquitectura de três camadas:

- Interface do Utilizador (main.py): Menu de texto no terminal que recebe as acções do utilizador.
- Lógica de Negócio — BLL (clientes.py, quartos.py, reservas.py, checkout.py, relatorios.py, auth.py): Implementa as regras do hotel.
- Acesso a Dados — DAL (database.py): Responsável pela criação e gestão da base de dados SQLite.

Design de Componentes

Ficheiro	Camada	Responsabilidade
database.py	DAL	Criação da BD, tabelas e utilizador admin inicial
auth.py	BLL	Autenticação de utilizadores com hash de password
quartos.py	BLL	CRUD de quartos e consulta de disponibilidade
clientes.py	BLL	Registo, listagem, pesquisa e histórico de clientes
reservas.py	BLL	Efectuação, cancelamento e listagem de reservas
checkout.py	BLL	Registo de check-in e check-out com actualização de estados
relatorios.py	BLL	Geração de relatório de ocupação e receita
main.py	Interface	Menu principal que liga todos os módulos

Base de Dados

A base de dados SQLite («hotel.db») é composta por 4 tabelas:

- utilizadores — Armazena os dados de login e perfil de cada utilizador do sistema.
- clientes — Regista os dados dos hóspedes (nome, email, contacto, documento).
- quartos — Contém os quartos do hotel com tipo, preço por noite e estado.
- reservas — Liga clientes a quartos com as datas de estadia e estado da reserva.

O fluxo de estados de uma reserva:

confirmada → checkin → checkout

confirmada → cancelada (se cancelada antes do check-in)

6. Implementação

Cronograma — Diagrama de Gantt (25 horas)

Fase	Tarefa	Sessão	Horas
Fase 1 – Requisitos	Levantamento de requisitos	S1	H1–H2
	Definição do âmbito do projeto	S1	H3
	Análise de viabilidade	S1	H4–H5
Fase 2 – Arquitectura	Diagrama de arquitectura	S2	H6–H7
	Modelação da base de dados	S2	H8–H9
	Protótipo das interfaces	S2	H10
Fase 3 – Desenvolvimento	Gestão de clientes	S3	H11
	Gestão de quartos	S3	H12–H13
	Sistema de reservas	S3	H14–H15
	Módulo Check-in e Check-out	S3/S4	H16–H18
Fase 4 – Testes	Testes unitários	S4	H19–H20
	Testes de integração	S4	H21
	Correção de erros	S4	H22
Fase 5 – Apresentação	Preparação da apresentação	S5	H23–H24
	Elaboração do relatório	S5	H25
	Entrega final	S5	H25

Tecnologias Utilizadas

- Linguagem de Programação: Python 3.10+
- Base de Dados: SQLite3 (incluído na biblioteca padrão do Python)
- Módulos Python: sqlite3, hashlib, datetime
- Ferramentas: Visual Studio Code, Git, GitHub Desktop

Estrutura de Código

- main.py — Ponto de entrada e menu principal do sistema
- database.py — Criação e gestão da base de dados
- auth.py — Autenticação de utilizadores
- quartos.py — Gestão de quartos

- clientes.py — Gestão de clientes
- reservas.py — Gestão de reservas
- checkinout.py — Check-in e check-out
- relatorios.py — Relatórios de ocupação

Controlo de Versão

- Ferramenta utilizada: Git
- Plataforma: GitHub

7. Testes

Plano de Testes

- Testes Unitários: Teste de cada função individualmente.
- Testes de Integração: Verificação da integração entre módulos.
- Testes de Sistema: Teste do sistema completo em cenários reais de utilização.

Casos de Teste e Resultados

Caso de Teste	Descrição	Resultado
Registo de Cliente	Registar um cliente com dados válidos.	Sucesso
Email duplicado	Tentar registar um cliente com email já existente.	Sucesso — sistema recusa
Adicionar Quarto	Adicionar um quarto com número, tipo e preço.	Sucesso
Consultar Disponibilidade	Verificar quartos livres para um período.	Sucesso
Efetuar Reserva	Criar reserva associando cliente e quarto.	Sucesso
Reserva Duplicada	Tentar reservar quarto já ocupado no mesmo período.	Sucesso — sistema recusa
Cancelar Reserva	Cancelar uma reserva confirmada.	Sucesso
Check-in	Registar entrada do cliente.	Sucesso
Check-out	Registar saída e calcular valor total.	Sucesso
Login inválido	Tentar autenticar com password errada.	Sucesso — sistema recusa
Relatório de Ocupação	Gerar relatório para um período com reservas.	Sucesso

8. Implantação

Plano de Implantação

- Requisito: Python 3.10 ou superior instalado no computador.
- Instalação: Copiar a pasta do projeto para o computador de destino.
- Inicialização: Executar `python database.py` para criar a base de dados.
- Execução: Executar `python main.py` para iniciar o sistema.
- Login inicial: Username admin / Password admin123 (alterar após primeiro acesso).

Formação

- O sistema não requer formação técnica prévia.
- A interface de menu de texto é intuitiva e autoexplicativa.
- O presente relatório serve como documentação de referência.

Suporte

- O código está organizado em módulos independentes, facilitando a manutenção.
- Cada função está documentada com docstrings em português.

9. Conclusão

Resumo dos Resultados

O Sistema de Gestão de Reservas de Hotel foi desenvolvido com sucesso, implementando todas as funcionalidades definidas nos requisitos (RF01 a RF11). O sistema permite gerir clientes, quartos e reservas de forma eficiente, eliminando os erros associados à gestão manual e garantindo a consistência dos dados através de validações e transações.

Próximos Passos

- Desenvolver uma interface gráfica (GUI) com Tkinter ou aplicação web.
- Adicionar gestão de pagamentos e emissão de facturas.
- Migrar para MySQL para suportar múltiplos utilizadores em simultâneo.
- Implementar sistema de notificações por email.

Reflexão

Este projeto permitiu consolidar os conhecimentos de Python e bases de dados adquiridos ao longo da formação. A arquitectura DAL/BLL demonstrou ser uma abordagem eficaz para organizar o código, facilitando a manutenção e a adição de novas funcionalidades. A maior dificuldade encontrada foi a validação das datas para evitar reservas duplicadas, resolvida com uma consulta SQL que verifica a sobreposição de períodos.

10. Anexos

- Documento de Levantamento de Requisitos (RF e RNF) — Ficheiro Excel UFCD 10790, versão 1.0, 02/06/2026
- Cronograma do Projeto (Diagrama de Gantt) — 25 horas distribuídas por 5 sessões
- Código fonte completo (repositório GitHub)
- Planos de Teste e Resultados
- Definição do Projeto — Tarefa 1, UFCD 10790